

From Mac M4 prototype to a device you can install in a store

Three connected stretches of work: prove the logic on your M4, prove the numbers on real RK3588 hardware, then lock the production device and run the Bengaluru pilot. Each phase lists exactly what to do next.

00 How to read this	01 Timeline	02 M4 prototype
03 Hardware gate	04 Production build	05 Device model
06 Open items		

00 How to read this plan

Three tracks, run mostly back-to-back with one deliberate overlap:

- TRACK A

 — everything you can build and test today on the Mac M4: recognition logic, SKU onboarding, the lookalike-rice model, the checkout flow. This proves the *software is correct*.
- TRACK B

 — the same pipeline ported onto an actual RK3588 board. This proves the *numbers are real*, since M4 latency does not predict RK3588 latency. It ends in a go/no-go gate before you commit to a custom board.
- TRACK C

 — the production device: carrier board, packing mechanics, certification, cloud backend, and the live pilot.

Track B starts in week 3, once Track A has something worth porting — you don't wait for Track A to fully finish.

01 12-week timeline

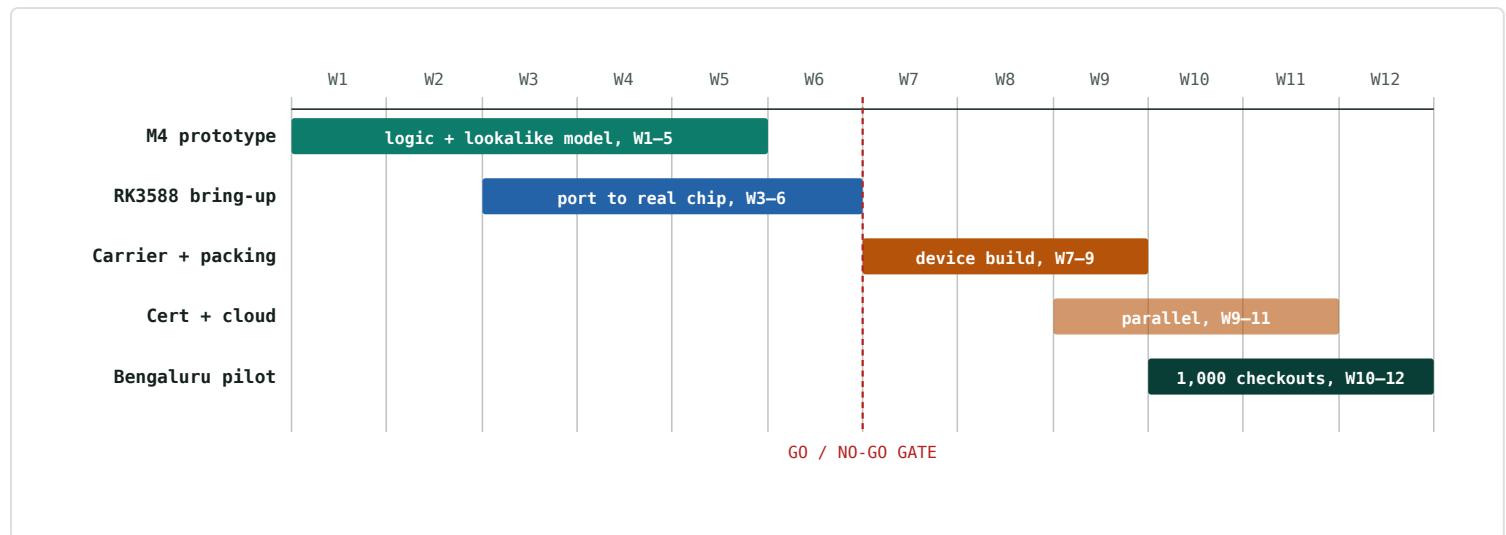


Fig 1 – Track A and B overlap weeks 3–6; production work (Track C) only begins once the week-6 gate is cleared.

02 TRACK A Local M4 prototype

Goal: prove the *logic* is correct — recognition, onboarding, promotions, offline resilience — using the Mac M4 as a stand-in compute box. None of the latency numbers from this track are final; they're a baseline to compare against Track B.

A0 · Environment setup

Day 1–2

- ❑ **Create the ARM64 conda environment** and install MLX, PyTorch+MPS, FAISS-CPU, OpenCV, transformers, mlx-vlm — this is the same stack your blueprint already scripted.
- ❑ **Load Qwen2-VL-2B (4-bit) via mlx-vlm** once, just to confirm the model loads and generates on the M4's unified memory.

A1 · Bridge real sensors into the M4

Day 2–4

- ❑ **Wire a TI ADS1232 breakout + load cell to a spare ESP32**
stream weight readings to the Mac over USB-serial (pyserial) — testing against real sensor noise beats testing against a hand-typed number every time.

- ❑ **Get one or two USB UVC cameras (or a capture card feeding the actual M12 lens modules)**

the M4 has no MIPI-CSI input, so this is the stand-in for the dual Sony IMX415/OV5647 cameras until the carrier board exists.

A2 · Core recognition pipeline

Week 1–2

- ❑ **Weight-gate trigger**

fire only on $\Delta W > 15g$ from the live serial stream.

- ❑ **Weight pre-filter**

narrow your local SKU catalog (JSON) to items within tolerance of the measured weight.

- ❑ **Fast path**

a lightweight pretrained embedding model (MobileNetV3/V4 via PyTorch+MPS) turns the live camera frame into a vector, matched against a local FAISS index.

- ❑ **Deep path**

Qwen2-VL fallback when the fast path isn't confident.

- ❑ **Log everything**

which path fired, confidence, per-stage timing — into a local SQLite table. This becomes your Track A benchmark report.

A3 · SKU onboarding tool

Week 2

- ❑ **Build a small capture script**

4 photos + weight per SKU → embedding → append to the local FAISS index + catalog metadata.

- ❑ **Onboard 15–20 test SKUs**

mix packaged snacks, dals, and all four rice types — you need a realistic catalog before the next phase means anything.

A4 · Lookalike-rice model

Week 3–4

- ❑ **Capture 50–100 labeled photos per rice variety**

Basmati, Sona Masoori, plain rice, idli rice — vary fill level, angle, lighting.

- ❑ **Add the density feature**

weight ÷ estimated tray-fill volume, computed alongside the image.

- ❑ **Baseline test**

run the off-the-shelf embedding + density fusion, build a confusion matrix across the four varieties.

❑ **Fine-tune if the baseline is weak**

contrastive/triplet fine-tuning on the captured set, still on the M4 via PyTorch+MPS. Target: >95% top-1 accuracy among the four varieties before moving on.

A5 · Checkout flow + promotions simulation

Week 4

❑ **Minimal local web UI**

weigh → identify (show candidate + confidence) → apply a promotions JSON rule file → mock payment → print a "pack" signal.

❑ **Exercise discount and free-item rules**

% off, buy-X-get-Y, ₹0 loyalty override — against the same rule format planned for production.

A6 · Offline resilience + benchmark writeup

Week 5

❑ **Local outbox table for transactions**

plus a mock "cloud" endpoint (can be a script on your own machine) to prove the sync pattern.

❑ **Kill the mock endpoint mid-test**

confirm the queue holds and drains correctly once it's back.

❑ **Write the Track A report**

accuracy numbers, model sizes, tuned confidence thresholds. This is what hands off to Track B — not the M4 latency numbers themselves.

03

TRACK B

Real-hardware validation gate

This track exists because an M4 has roughly 5–6x the RK3588's NPU throughput. Nothing you measured in Track A tells you whether the real device will hit <350ms — only the real chip does.

B1 · Get a real board

Week 3–4, parallel with A4

❑ **Order an Orange Pi 5 Plus or Radxa Rock 5B**

RK3588, 8GB+ RAM — roughly \$100–150, and the fastest way to get real numbers without waiting on the custom carrier board.

❑ **Install RKNN-Toolkit2**

set up the quantization/conversion environment on a dev machine.

B2 · Port the models

Week 5–6

❑ **Convert the fast-path embedding model to INT8 via RKNN-Toolkit2**

budget 1–3 engineer-days for calibration and accuracy validation.

❑ **Convert the VLM to INT4 via the RKLLM toolkit**

this is the least-proven step in the whole plan — start it early, expect iteration.

❑ **Re-run the identical test set from Track A**

same photos, same weights, same rice-variety dataset — so the comparison is apples-to-apples.

B3 · Real latency + thermal test

Week 6

❑ **Measure per-stage latency on the RK3588 board**

fast path, deep path, end-to-end — against the <30ms / ~250ms / <350ms targets.

❑ **Run a sustained load test**

e.g. 500 back-to-back checkouts inside a sealed test box, not an open dev board — this is what actually reveals thermal throttling.

B4 · Go / no-go decision

End of Week 6

GATE

❑ **Go**

latency and thermal numbers hold up → lock RK3588S, start the custom carrier board (Track C).

❑ **No-go**

if quantization can't close the gap, this is the one point in the plan where reopening the RK3588-vs-Jetson decision is worth the cost — not before.

C1 · Carrier board + packing mechanics

Week 7–9

❑ Finalize the custom carrier board

routes dual MIPI-CSI (cameras), eDP (touch panel), RS-485 (ADS1232 weight ADC), and a UART/CAN handoff to the packing MCU.

❑ Confirm the camera modules

Sony IMX415/OV5647 with M12 fixed-barrel lenses and bonded polarizers, per the blueprint.

❑ Put packing on its own MCU

a second ESP32 (or STM32) drives the drop + seal actuators for the 20kg chute — kept off the RK3588 so a burst of AI inference never makes the packing arm stutter, and vice versa.

❑ Resize the chassis

CRCA steel, IP52, now sized for compute + packing mechanism together, with passive airflow paths so the RK3588 stays fanless.

❑ Size the power supply

confirm the MeanWell 12V/5A budget still covers compute + cameras + touch panel + packing actuators, or step up if the motors draw more.

C2 · BOM & manufacturing partner

Week 7–9, parallel

❑ Get the packing subsystem quoted

motors, actuators, chute, sealing consumables — this is the piece missing from the blueprint's ₹32.8–38.5K checkout-only BOM.

❑ Engage a prototype/manufacturing partner

the same way Smart Soak used Inceptarc — either that vendor pool or one that specializes in kiosk-class hardware.

❑ Reconcile the target BOM

toward the deck's ₹98.5K batch figure once packing is quoted for real.

C3 · Certification

Week 9–11, parallel

❑ Legal Metrology certification

required — the device bills customers by weight.

❑ BIS & WPC approvals

as already scoped in the blueprint.

C4 · Cloud & fleet software

Week 7–10, parallel

❑ **Stand up the brand/branch/kiosk hierarchy backend**

master catalog in Qdrant, MQTT broker, analytics store, promotions engine.

❑ **Build the admin dashboard**

SKU onboarding, branch config, promotions, daily checkout analytics per device.

❑ **Build the OTA pipeline**

versioned model/software updates with rollback.

C5 · Bengaluru pilot

Week 10–12

❑ **Install the pilot unit(s)**

first real store deployment.

❑ **Run to the 1,000-checkout target**

the milestone that matters more than a polished machine.

❑ **Watch five things closely**

recognition accuracy on lookalikes, real-world latency, thermal behavior over a full day, payment reliability, and packing mechanism safety.

❑ **Feed pilot data back**

into the lookalike model and the final BOM before scaling to a second branch.

05 Production device model

What ships in the box, subsystem by subsystem:

AutoDock & Pack – production hardware model

SUBSYSTEM	SPEC	COST BASIS
Compute core	RK3588S SOM (8GB) + custom carrier board	₹9,800 (blueprint)
Vision	Dual Sony IMX415 / OV5647, M12 lens, bonded polarizers	₹4,800 (blueprint)
Weighing	TI ADS1232 24-bit ADC + 4 load cells, $\Delta W > 15\text{g}$ trigger	₹1,100 (blueprint)
Checkout UI	10.1"/13.3" IPS eDP touch panel	₹6,800 (blueprint)
Chassis	CRCA steel, powder-coated, IP52, resized for packing	₹7,500+ (blueprint, pre-resize)

Power	MeanWell 12V/5A open-frame PSU, surge protection	₹2,800 (blueprint)
Packing subsystem	Dedicated MCU (ESP32/STM32) + drop & seal actuators, 20kg-rated chute	TBD – get vendor quote
Connectivity	WiFi/Ethernet + optional 4G fallback, MQTT client	TBD – get vendor quote

Checkout-only subsystems total ₹32,800–38,500 per the blueprint. Packing and connectivity are the two line items driving the deck's higher ₹98.5K batch BOM — get both quoted during C2 before treating that number as final.

06 Open items to close before you commit hardware spend

UNQUOTED

Packing subsystem cost (motors, actuators, chute, sealing consumables) — nothing in either source document prices this out yet.

UNVALIDATED

Every latency number in this plan (<30ms / ~250ms / <350ms) is a target until Track B, Phase B3 produces real RK3588 numbers under thermal load.

UNCONFIRMED

Manufacturing partner for the kiosk hardware — Inceptarc is your Smart Soak vendor; decide whether they (or someone new) build this too.

WORTH DECIDING EARLY

Whether customers load items one at a time or in a mixed batch — that answer changes whether the fast path needs object detection (YOLO-class) or can stay a single-item classifier, per the earlier tech-stack discussion.

